

MILITARY ASSET MANAGEMENT SYSTEM

FINAL DOCUMENTATION & TECHNICAL REPORT

Architecture • ER Schema • API Reference • RBAC • Setup • Test Accounts

Project	Military Asset Management System
Frontend	React + Vite + Tailwind CSS + Recharts
Backend	Node.js + Express.js
Database	PostgreSQL
Security	JWT + Bcrypt + RBAC
Testing	Postman + browser-based functional testing

Prepared for assignment submission

Based on the supplied project requirements and the implementation files verified during development.

1. Executive Overview

The Military Asset Management System is a role-based inventory platform for managing military equipment across multiple bases. It supports authentication, current stock visibility, purchases, transfers, assignments, expenditures and audit logging.

- Track equipment quantities by base and equipment type.
- Calculate dashboard movement metrics and closing balance.
- Support cross-base transfers using transactional database operations.
- Restrict Base Commanders to their assigned base.
- Provide an audit trail for asset-changing operations.
- Provide Dashboard filters for Base, Equipment Type, Start Date and End Date.

The supplied reference defines the core objective, transaction model, RBAC and dashboard requirements.

■filecite■turn2file0■L5-L15■

2. Overall Architecture

Presentation: React/Vite SPA → **API:** Axios → Express REST API → **Security:** JWT + RBAC middleware → **Business Logic:** controllers → **Persistence:** PostgreSQL

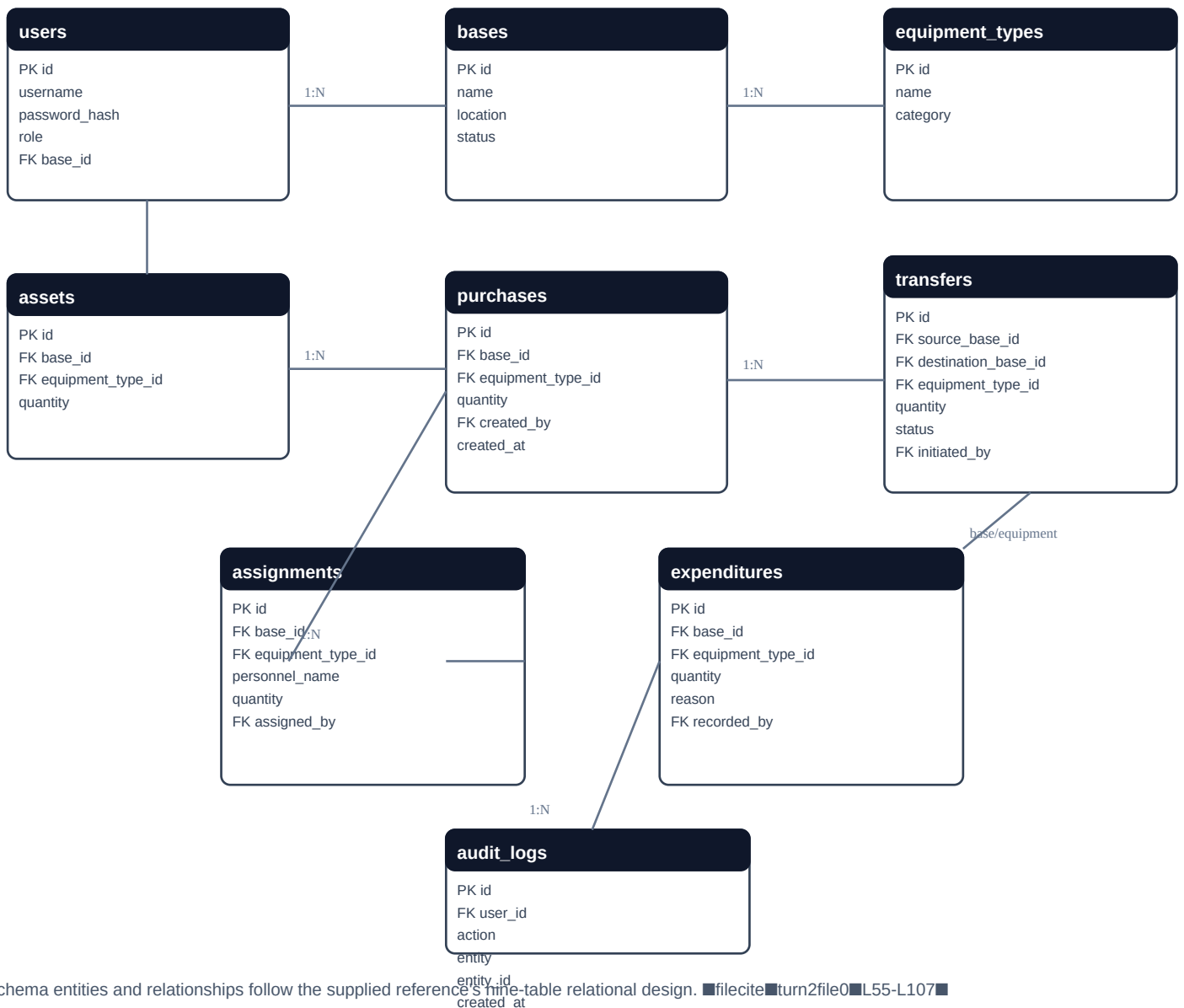
Layer	Components	Responsibility
Frontend	React, React Router, Tailwind, Recharts, Axios	UI, navigation, forms, dashboard, API calls
Authentication	AuthContext + JWT	Login state and authenticated requests
API	Express routes + controllers	REST endpoints and business rules
Middleware	JWT authentication + base-scope/RBAC	Authentication and authorization
Database	PostgreSQL	Master data, current assets, transactions, audit history
Transactions	PostgreSQL BEGIN/COMMIT/ROLLBACK	Atomic cross-base transfers

The reference requires authentication middleware, RBAC middleware and atomic transfer handling.

■filecite■turn2file0■L111-L181■

3. ER Schema Diagram

The following diagram summarizes the nine core relational tables and their principal relationships. PK = primary key; FK = foreign key.



4. Data Model & Inventory Logic

Entity	Role in System
users	Identity, role and base assignment
bases	Military base master data
equipment_types	Equipment master data and category
assets	Current stock quantity per base/equipment
purchases	Positive stock movements
transfers	Completed source/destination stock movements
assignments	Stock allocated to personnel
expenditures	Stock consumed for a reason
audit_logs	Traceability of mutation events

Net Movement = Purchases + Transfers In – Transfers Out

Closing Balance = Opening Balance + Net Movement – Assigned – Expended

The verified metrics controller applies date, base and equipment filters to purchases, completed transfers, assignments and expenditures.

5. Detailed REST API Endpoint Listing

Method	Endpoint	Auth	Purpose / Request Data
GET	/api/health	Public	Health check; returns status ok
POST	/api/auth/login	Public	Username/password login; returns authenticated session/token
GET	/api/bases	JWT	List base master data
GET	/api/equipment-types	JWT	List equipment type master data
GET	/api/assets/metrics	JWT + scope	Metrics; query: baseId, equipmentTypeId, startDate, endDate
GET	/api/assets/stock	JWT + scope	Current stock; query: baseId, equipmentTypeId
GET	/api/purchases	JWT/RBAC	List purchase records
POST	/api/purchases	JWT/RBAC	Create purchase: baseId, equipmentTypeId, quantity
GET	/api/transfers	JWT/RBAC	List transfer records
POST	/api/transfers	JWT/RBAC	Create transfer: sourceBaseId, destinationBaseId, equipmentTypeId, quantity
GET	/api/assignments	JWT/RBAC	List assignment records
POST	/api/assignments	JWT/RBAC	Create assignment: baseId, equipmentTypeId, personnelName, quantity
GET	/api/expenditures	JWT/RBAC	List expenditure records
POST	/api/expenditures	JWT/RBAC	Create expenditure: baseId, equipmentTypeId, quantity, reason
GET	/api/audit-logs	JWT/RBAC	List audit records and mutation history

The verified asset routes apply authentication and base-scope middleware before metrics and stock.

Dashboard Metrics Query Parameters

Parameter	Type	Meaning
baseId	integer	Filter metrics to a base; backend scope can override it
equipmentTypeId	integer	Filter to an equipment type
startDate	timestamp/date	Include transactions from this point
endDate	timestamp/date	Include transactions through this point

The metrics controller explicitly reads these query parameters and uses them in the SQL aggregates.

6. RBAC Authorization Matrix

Capability	ADMIN	BASE COMMANDER	LOGISTICS OFFICER
Global visibility	✓	No	As permitted
Assigned-base scope	All bases	Own base only	Operational access
Purchases	✓	✓ base scoped	✓
Transfers	✓	Restricted	✓
Assignments	✓	✓ base scoped	—
Expenditures	✓	✓ base scoped	—
Audit Logs	✓	Scoped as applicable	As permitted

The assignment reference defines Admin as unrestricted, Base Commander as assigned-base scoped, and Logistics Officer as primarily responsible for Purchases and Transfers. █filecite█turn2file0█L137-L163█

7. Security & Transaction Controls

- JWT-based authentication.
- Bcrypt password hashing.
- Helmet and CORS configuration.
- Protected API routes.
- Base-scope authorization.
- Minimum quantity validation (1).
- Insufficient-stock validation.
- Atomic transfer transaction with rollback on failure.
- Audit records for asset-changing operations.

These controls align with the reference security and transaction requirements. █filecite█turn2file0█L111-L181█

8. Setup & Run Instructions

PostgreSQL

- Install PostgreSQL and pgAdmin.
- Create the project database.
- Run supplied schema and seed SQL.
- Verify master data and initial assets.

Backend

```
cd backend
npm install
npm run seed:users
npm run dev
```

Configure .env with PORT, DATABASE_URL, JWT_SECRET and CLIENT_URL. Keep secrets out of Git.

Frontend

```
cd frontend
npm install
npm run dev
```

Open the Vite development URL shown in the terminal and sign in using a test account.

9. Sample Test Accounts

Role	Username	Password	Expected Scope
ADMIN	admin_user	AdminPass123!	All bases and full administrative access
BASE_COMMANDER	commander_alpha	CommandPass123!	Fort Alpha / assigned base
LOGISTICS_OFFICER	logistics_officer	LogisticsPass123!	Purchases and Transfers focus

These credentials follow the supplied assignment sample account section. █filecite█turn2file0█L339-L345█

10. Functional & Negative Testing

Scenario	Result
Admin login	PASS
Base Commander login and Fort Alpha scope	PASS
Purchase creation and stock update	PASS
Transfer and source/destination update	PASS
Assignment and stock reduction	PASS
Expenditure and stock reduction	PASS
Audit logging	PASS
Quantity 0	Rejected by minimum-value validation
Assignment above available stock	Rejected: Insufficient stock
Expenditure above available stock	Rejected: Insufficient stock
Transfer above available source stock	Rejected: Insufficient stock
Dashboard Base/Equipment filters	PASS
Dashboard Date filters	PASS

11. Required Submission Artifacts

Artifact	Contents / Status
Clean Source ZIP	Frontend + backend + database + docs; exclude node_modules
Documentation PDF	This final report, including ER schema and detailed API listing
Required Screenshots	Dashboard, operations, audit and RBAC views
Video Demo	3–5 minute architecture + live functional walkthrough + RBAC switching
Live Frontend	Netlify or Vercel URL after deployment
Live Backend	Render or Railway URL after deployment

The supplied checklist requires the source archive, documentation PDF, video walkthrough and live links.

filecite turn2file0 L303-L337

12. Project Structure

```

military-asset-management/
├── backend/
│   ├── config/
│   ├── controllers/
│   ├── middlewares/
│   ├── routes/
│   ├── scripts/
│   └── server.js
├── frontend/
│   ├── src/
│   ├── components/
│   ├── context/
│   ├── pages/
│   ├── services/
│   └── App.jsx
├── database/
├── docs/
├── README.md
└── .gitignore

```

13. Final Notes

Core application workflows and negative validation scenarios were tested during development. Final submission preparation should include screenshot capture, the 3–5 minute demo recording, frontend/backend deployment, verified live URLs and creation of the clean final ZIP.

No public deployment URL is claimed in this document because the verified environment is local development.